Polynomials and FFT

Part VII Selected Topics

Polynomial multiplication becomes pointwise multiplication after evaluation at distinct points. We derive the DFT and its inverse from root orthogonality, then check the even/odd split, bit reversal, butterfly indices, and zero padding. Exact arithmetic and floating-point arithmetic require different guarantees.

Prerequisites are complex roots, value representations, and recursive divide and conquer.

Lin Cing
https://github.com/xinhi6716/computer-science-self-stu

Chapter map

- Compare multiplication in coefficient and value representations.
- Derive the DFT and inverse DFT.
- Implement radix-2 FFT and state the conditions on the roots of unity.

The sections cover polynomial representations, the DFT and FFT, and efficient FFT implementations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Coefficient representation

A polynomial $A(x) = \sum_{j=0}^{n-1} a_j x^j$ has **degree bound** n, meaning $\deg A < n$. Its leading coefficients may be zero, so its actual degree may be smaller. Code may represent the zero polynomial by `[0]`. The degree bound is not the location of the first nonzero entry. Coefficients here run from low degree to high degree.

Addition of equal-length coefficient vectors takes $\Theta(n)$. Horner evaluation at z starts with $v = a_{n-1}$ and updates $v = a_j + zv$ for $j = n-2, \ldots, 0$. After the update at j, the invariant is $v = \sum_{r=j}^{n-1} a_r z^{r-j}$. The result is $A(z)$, using $n-1$ multiplications and additions. Independent Horner evaluations at n arbitrary points cost $\Theta(n^2)$.

For input lengths r, s, convolution gives $c_k = \sum_{i+j=k} a_i b_j$ and output length $r + s - 1$. Direct convolution uses rs coefficient products, or $\Theta(rs)$ operations. FFT improves the entire conversion-and-multiplication process.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Interpolation exists and is unique

Given distinct $x_0, \ldots, x_{n-1}$ and values $y_0, \ldots, y_{n-1}$, define

$$\ell_k(x) = \prod_{j \neq k} \frac{x - x_j}{x_k - x_j}, \qquad A(x) = \sum_k y_k \ell_k(x).$$

Every denominator is nonzero. Each basis polynomial has degree less than n and satisfies $\ell_k(x_i) = \delta_{ik}$, proving existence.

If two polynomials of degree less than n match the values, their difference has n distinct roots. A nonzero polynomial of degree d has at most d roots. To prove this, factor out $x - \alpha$ at a root α. Every other root is a root of the quotient, so induction on degree applies. The difference must be zero. With fewer than n points, adding any multiple of $\prod_j (x - x_j)$ over the known points usually gives distinct solutions within the degree bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Vandermonde matrices and interpolation cost

Write $V[i,j] = x_i^j$ and $Va = y$. The Vandermonde determinant is $\det V = \prod_{j<k}(x_k - x_j)$. Equal points give equal rows, so every difference is a factor of the determinant polynomial. Its total degree is exactly $n(n-1)/2$. The coefficient of $x_1 x_2^2 \cdots x_{n-1}^{n-1}$ is 1, fixing the constant factor. Distinct points therefore make V invertible. General LUP interpolation costs $\Theta(n^3)$.

Lagrange interpolation can be converted to coefficients in $O(n^2)$ arithmetic operations. Build $Q(x) = \prod_j (x - x_j)$ one factor at a time in total quadratic time. For each k, use linear-time synthetic division to obtain $Q/(x - x_k)$, evaluate its denominator in $O(n)$, and accumulate coefficients. Rebuilding every basis polynomial from scratch does not justify that quadratic bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A product needs enough values

At the **same N distinct points**, multiplication needs only N pointwise products. Unique reconstruction of lengths r, s requires $N \geq r + s - 1$. Pad both coefficient arrays to N. Radix-2 FFT chooses a power of two at least this large.

For $A = 1 + 2x$ and $B = 1 - x$, the product is $1 + x - 2x^2$, which needs three coefficients. Choose $N = 4$, even though each input has length 2.

At the Nth roots of unity, $x^N = 1$. With too few points, the inverse returns convolution modulo $x^N - 1$. Position k receives $c_k + c_{k+N} + c_{k+2N} + \cdots$. For $(1 + x)^2$, $N = 2$ gives `[2,2]` because the x^2 coefficient wraps into the constant coefficient. Ordinary multiplication gives `[1,2,1]`. Zero padding prevents this aliasing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Roots of unity and the forward DFT

Use the **positive-exponent forward convention** throughout:

$$\omega_n = e^{2\pi i/n}, \qquad y_k = \sum_{j=0}^{n-1} a_j \omega_n^{jk} = A(\omega_n^k).$$

Some fields use a negative forward exponent. Either paired convention works, but mixing them does not.

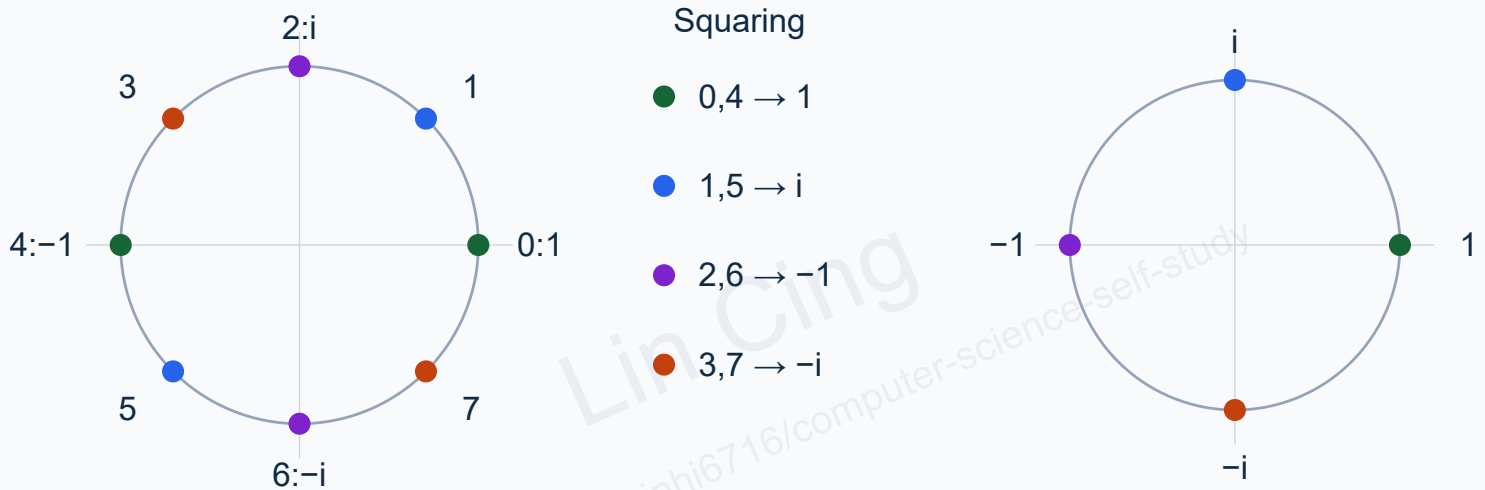
The root is primitive, so $\omega_n^k = 1$ exactly when $n \mid k$. Its n powers are distinct interpolation points. Cancellation gives $\omega_{dn}^{dk} = \omega_n^k$ for positive integers d. For even n, $\omega_n^{n/2} = -1$ and $(\omega_n^k)^2 = \omega_{n/2}^k$. Opposite roots square to the same value, leaving $n/2$ distinct roots, each twice.

The geometric sum $\sum_{j=0}^{n-1} \omega_n^{jt}$ is n if $n \mid t$. Otherwise it is $(1 - (\omega_n^t)^n)/(1 - \omega_n^t) = 0$. Primitivity makes the denominator nonzero in the latter case. The value 1 cannot replace a primitive root for arbitrary n.

Roots of unity and the forward DFT (continued)

Eight roots square to four distinct points

Left label k denotes ω_8^k. Same-color opposite points square to one point on the right.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Orthogonality gives the inverse

Let $F[k, j] = \omega_n^{jk}$ and $G[j, k] = \omega_n^{-jk}/n$. Then

$$(GF)[j, \ell] = \frac{1}{n} \sum_k \omega_n^{k(\ell - j)} = \delta_{j\ell}.$$

For distinct $j, \ell \in [0, n-1]$, $n \nmid \ell - j$, so orthogonality applies. Thus $GF = I$, $G = F^{-1}$, and

$$a_j = \frac{1}{n} \sum_k y_k \omega_n^{-jk}.$$

Use negative twiddles and divide by n once at the end. Normalization may instead be distributed across levels, but dividing by n at every level is wrong.

Applying the forward transform twice and dividing by n gives $(F^2 a/n)_j = a_{(-j) \bmod n}$. This is cyclic index reversal, not conjugation for arbitrary complex input. For $a = [2, 5+i, 7, -1]$, the result is `[2,-1,7,5+i]`.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Even and odd subproblems

For a positive power of two n, set $A_e(z) = a_0 + a_2 z + \cdots$ and $A_o(z) = a_1 + a_3 z + \cdots$. Then $A(x) = A_e(x^2) + xA_o(x^2)$. Root halving reduces evaluation at all n roots to two $n/2$-point DFTs, E and O.

For $0 \le k < n/2$,

$$y_k = E_k + \omega_n^k O_k, \qquad y_{k+n/2} = E_k - \omega_n^k O_k.$$

Compute $t = \omega_n^k O_k$ once, then add and subtract. This is a butterfly. The base case $n = 1$ returns `[a[0]]`. Assuming the two recursive transforms are correct, the displayed identities produce every required value, proving correctness by induction.

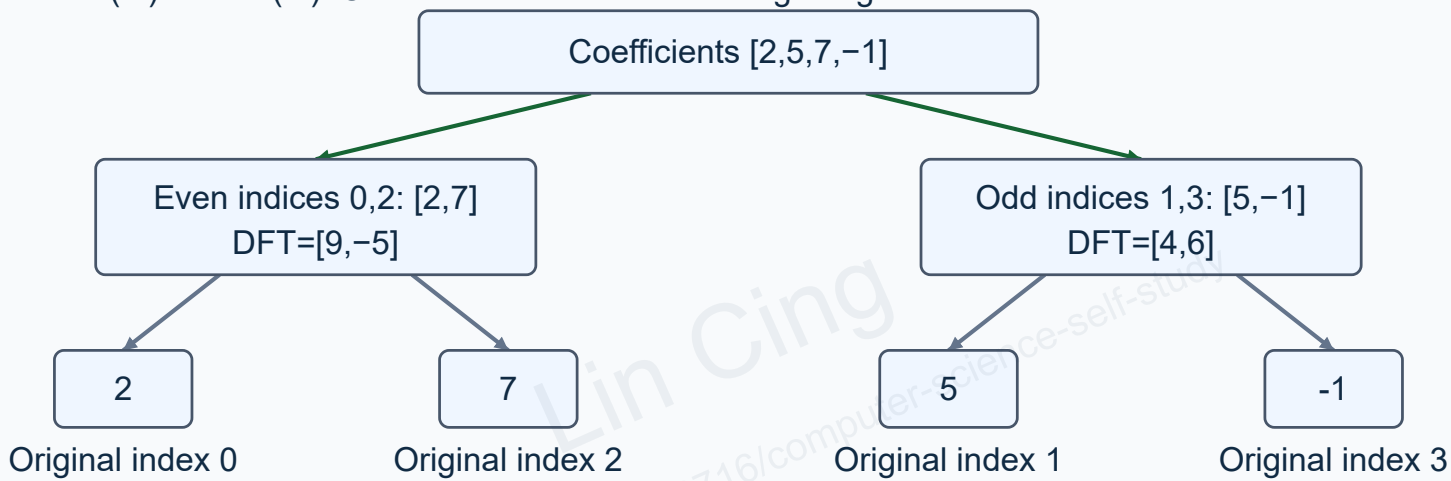
For `[2,5,7,-1]`, the even coefficients `[2,7]` give `[9,-5]`, and the odd coefficients `[5,-1]` give `[4,6]`. Twiddle 1 gives $y_0 = 13, y_2 = 5$. Twiddle i gives $y_1 = -5 + 6i, y_3 = -5 - 6i$. Distinct intermediate values expose incorrect wiring.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Even and odd subproblems (continued)

Even/odd split: subtransforms and leaf indices

$A(x) = A_{even}(x^2) + xA_{odd}(x^2)$. Coefficients run from low to high degree.



Left-to-right leaf indices 0,2,1,3 are the two-bit reversal order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Recursive FFT and its cost

```
FFT(a, sign):
    n = len(a)                    # require a positive power of two
    if n == 1: return [a[0]]
    E = FFT([a[0], a[2], ...], sign)
    O = FFT([a[1], a[3], ...], sign)
    w = 1; wm = exp(sign * 2*pi*i/n)
    for k = 0 .. n/2-1:
        t = w * O[k]
        y[k] = E[k] + t
        y[k+n/2] = E[k] - t
        w = w * wm
    return y
```

Lin Cing
github.com/xinhi6716/computer-science-self-study

Forward uses `sign=+1`. Inverse uses `sign=-1` and divides the outer result by n. Splitting and combining cost $\Theta(n)$ per level. Thus $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$ complex arithmetic operations. There are $n/2$ butterflies per level and $(n/2) \log_2 n$ in total.

Recursive FFT and its cost (continued)

A sequential implementation that copies subarrays and promptly releases child results needs $O(n)$ peak extra space and $O(\log n)$ stack space. Retaining all levels takes $O(n \log n)$. Recursion depth alone does not bound the arrays to logarithmic space.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

All three transforms in a convolution

For $A = [1, 2, 0, 0]$, $B = [1, -1, 0, 0]$, $N = 4$, and $\omega_4 = i$:

k	DFT(A)	DFT(B)	Pointwise product
0	3	0	0
1	$1 + 2i$	$1 - i$	$3 + i$
2	-1	2	-2
3	$1 - 2i$	$1 + i$	$3 - i$

The negative-exponent DFT of `[0,3+i,-2,3-i]` is `[4,4,-8,0]` before normalization. Dividing by 4 gives `[1,1,-2,0]`. Keep the first three coefficients to recover $1 + x - 2x^2$.

Two forward transforms, one inverse, and $\Theta(N)$ pointwise products cost $\Theta(N \log N)$. Because the output degree is below N, cyclic convolution equals ordinary convolution.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

All three transforms in a convolution (continued)

Convolution: frequency values and inverse

$A=1+2x$, $B=1-x$, padded to $N=4$. Positive forward exponent, $\omega_4=i$.

k	DFT(A)	DFT(B)	Pointwise product
0	3	0	0
1	$1+2i$	$1-i$	$3+i$
2	-1	2	-2
3	$1-2i$	$1+i$	$3-i$

Negative DFT: $[4,4,-8,0]$. Divide by 4: $[1,1,-2,0]$.

Green cells are product values. Keep three coefficients: $1+x-2x^2$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bit reversal and stage 1

Recursive splitting reads the lowest index bit first, then the next bit. For $n = 2^b$, the leaf order reverses the b bits of each original index. Bit reversal is its own inverse, so both `A[rev(j)]=a[j]` and `A[j]=a[rev(j)]` describe the permutation.

For $n = 4$, `00,01,10,11` become `00,10,01,11`. The input `[2,5,7,-1]` becomes `[2,7,5,-1]`. Stage 1 pairs **physical positions** `(0,1)` and `(2,3)`, which hold original indices `(0,2)` and `(1,3)`. These are different kinds of index.

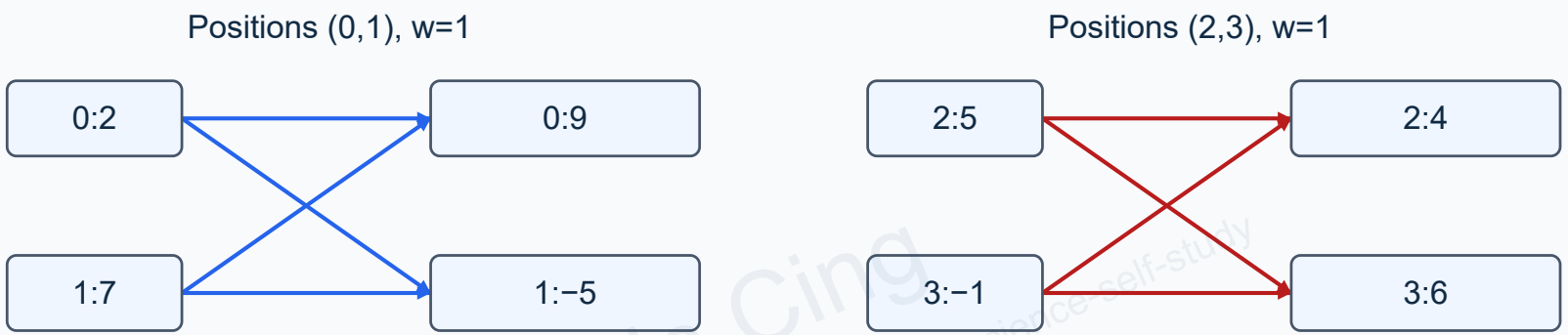
Both stage-1 twiddles equal 1. The array becomes `[9,-5,4,6]`. Stage 2 uses $m = 4$, pairs `(0,2)` and `(1,3)`, and uses twiddles 1 and i. Pairing adjacent positions again would perform a different computation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bit reversal and stage 1 (continued)

Stage 1: adjacent physical positions

Cells show position:value. Upper output u+wv; lower output u-wv.



Blue pair has original indices (0,2); red pair has original indices (1,3).

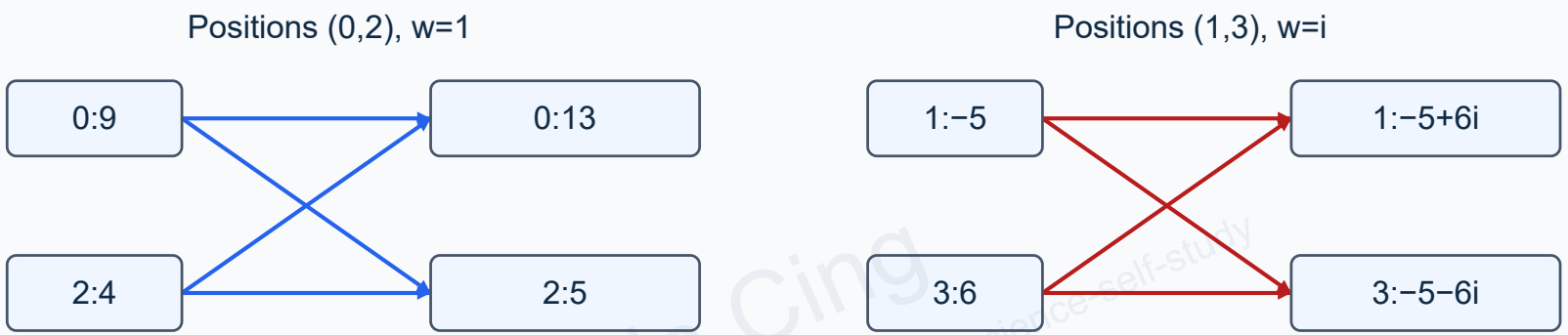
After m=2: [9,−5,4,6]. The next stage uses separation 2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stage 2 combines across half-blocks

Stage 2: half-block pairs and twiddles

Cells show position:value. Upper output u+wv; lower output u-wv.



Blue pair (0,2): 9±4. Red pair (1,3): −5±i·6.

After m=4, positions 0,1,2,3 hold [13,−5+6i,5,−5−6i].

j	$u = A[j]$	$t = \omega_4^j A[j+2]$	**New** $A[j]$	**New** $A[j+2]$
0	9	$1 \cdot 4 = 4$	13	5
1	-5	$i \cdot 6 = 6i$	$-5 + 6i$	$-5 - 6i$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Stage 2 combines across half-blocks (continued)

In physical output order, the result is `[13,-5+6i,5,-5-6i]`.

The same rules take `[1,2,0,-1]` through bit reversal `[1,0,2,-1]`, stage 1 `[1,1,1,3]`, and stage 2 `[2,1+3i,0,1-3i]`. Repeated intermediate 1s can conceal wiring mistakes. The earlier example makes the sources of each operation distinct.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Iterative FFT index contract

```
place a into A in bit-reversed order
for s = 1 .. log2(n):
    m = 2^s; wm = exp(sign * 2*pi*i/m)
    for base = 0, m, 2*m, ..., n-m:
        w = 1
        for j = 0 .. m/2-1:
            u = A[base+j]
            t = w * A[base+j+m/2]
            A[base+j] = u+t
            A[base+j+m/2] = u-t
            w = w*wm
for inverse only, divide every final entry by n
```

At the start of stage s, each length-m block has the DFT of its even subproblem in its left half and the DFT of its odd subproblem in its right half. Butterflies perform the recursive combination, leaving a complete m-point DFT in each block. Singletons establish the stage-1 base case. Induction reaches $m = n$. Save `u` before overwriting the upper cell, or the lower subtraction will use a new value.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Iterative FFT index contract (continued)

Each stage has $n/2$ butterflies, so arithmetic work is $\Theta(n \log n)$. Reversing b bits separately for each index takes $O(n \log n)$ bit steps. Reorder with an extra length-n array or in-place swaps of the permutation's two-cycles. Twiddles depend on m, j, not `base`. All blocks in a stage can share a table. Tables for all stages contain $\Theta(n)$ values and do not change the main arithmetic bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Circuit work and depth

With fixed twiddles and bit reversal treated as wires, a bounded-fan-in circuit runs all $n/2$ disjoint butterflies in a stage at once. Only the next stage reads their outputs. For $n = 8$, there are 3 stages and 12 butterflies. Work is $\Theta(n \log n)$, while depth is $\Theta(\log n)$, since every input-to-output path crosses $\log_2 n$ combinations.

This model assumes constant-depth complex addition and multiplication. It does not include serial twiddle generation, processor communication, or physical wire lengths. Software threads sharing an array need a barrier after each stage. Within a stage each butterfly writes a different pair of cells. With only P processors, the lower bound W/P still applies. A logarithmic-depth drawing does not give logarithmic running time on arbitrary hardware.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exact arithmetic and floating point

The $\Theta(n \log n)$ bound counts complex arithmetic operations. Floating-point roots and arithmetic introduce errors. Rounding an integer convolution is valid only with an absolute error below $1/2$ in every output cell. A few successful round trips cannot guarantee all lengths and coefficient magnitudes.

An exact alternative is an NTT over a prime field $\mathbb{F}_p$. Require $N \mid p - 1$ and a root of order exactly N. Then N is invertible and the same geometric-sum and inverse proofs apply. Over a composite modulus, a primitive root alone does not ensure that the required denominators are units. The statement here is restricted to prime fields.

For the same inputs, use $p = 17, N = 4, \omega = 13$. We have $\omega^2 = 16 \neq 1$, $\omega^4 = 1$, and $4^{-1} = 13$. The transforms are `[3,10,16,9]` and `[0,5,2,14]`. Pointwise multiplication modulo 17 gives `[0,16,15,7]`, and the inverse gives `[1,1,15,0]`. Decode 15 as -2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Signed reconstruction needs a magnitude bound

If all integer input coefficients have absolute value at most B, with lengths r, s, every convolution coefficient has absolute value at most $\min(r, s)B^2$. Choose $p > 2\min(r, s)B^2$ for unique signed reconstruction from one modulus. Here $r = s = 2, B = 2$, and $17 > 16$, so reconstruction is unique. Correct modular arithmetic alone cannot rule out wraparound of larger integers.

The NTT operation bound counts arithmetic modulo p. If p grows with coefficient magnitude, bit complexity must include multiplication and addition of $\log p$-bit integers. Arbitrarily large integer multiplication is not one constant-time operation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 30.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study